Emergency Training Simulator POC

Submission Packet

Roadmap

Project roadmap

Source: docs/roadmap.md

Prepared for review, discussion, and project handoff.

Markdown stays editable. This PDF is the clean reading copy.

Boundary: this is still a POC, not operational or clinical guidance.

Roadmap

This roadmap separates the current POC path from the full-system path.

That separation matters.

The full project vision is large. The current repo should grow toward it carefully, one trusted layer at a time.

Track 1: Current POC milestones

These are the milestones for this repository right now.

Completed

- Sprint 0 repo scaffolding
- deterministic backend scenario loop
- FastAPI WebSocket session endpoint
- JSON message contracts
- local JSONL session logs
- replay and review scripts
- one-command local demo
- wooden chair environment cue
- entry door environment cue
- future Unreal client contract
- text-only Unreal display preview
- first security baseline
- local demo abuse guardrails
- lightweight GitHub Actions CI
- UX docs for Dr. Cameron as first audience
- improved post-scenario review format
- project presentation package and PDF

Near-term

- keep improving the review/debrief output
- add richer instructor/reviewer examples
- strengthen the future Unreal text debug contract
- add clearer message-contract versioning notes
- add a small evaluation rubric for Dr. Cameron
- keep scenario logic deterministic and backend-owned

Later POC

- add another deterministic toy scenario
- add more environment objects with obvious state
- add richer replay comparison reports
- add configurable local scenarios
- add thin Unreal client experiment only after the text contract is stable

Track 2: Full-system milestones

These belong to the larger project vision.

They should not be treated as current repo features yet.

Future AI layer

- AI-assisted scenario generation
- agentic scenario adaptation
- model-based trainee action interpretation
- AI-supported review summaries
- validation of AI-generated outputs

Future validation layer

- structured medical/protocol references
- department-provided validated materials
- rule-based and model-based validation
- update process for changing guidelines
- review workflow for validation data

Future immersive environment layer

- Unreal scene development
- projection mapping experiments
- mannequin localization
- projected patient/environment changes
- synchronized audio/visual feedback

Future sensing layer

- voice input
- pose and gesture recognition
- motion tracking
- procedural action detection
- sensor confidence and uncertainty handling

Future deployment layer

- cloud services
- hub-and-spoke architecture
- Virginia Tech hub role
- spoke-site deployment model
- resilience and caching
- multi-site instructor support

Recommended build order

The safest build order is:

1. trusted backend loop
2. replay and review
3. message contracts
4. security and trust boundaries
5. UX and instructor review flow
6. future client display contract
7. richer deterministic scenarios
8. Unreal text/debug client
9. generated/reviewed asset pipeline
10. real physics experiments
11. sensing experiments
12. AI adaptation experiments
13. validation framework expansion

14. deployment architecture

Rule of thumb

If a feature changes scenario truth, validation, or review, build it in the backend first.

If a feature only displays state or captures input, it can live in a future client layer.

That keeps the system explainable as it grows.